



操作系统课程实验

——从0到1的小型内核实现

2025年 秋季学期

lab-9 文件系统完善与系统整合

1. 进程打开文件表
2. 设备文件
3. 用户库
4. 可执行文件
5. 总结

1. 进程打开文件表

linux当中万物皆文件，一个进程启动时会默认打开 3 个标准文件描述符：标准输入（stdin）、标准输出（stdout）、标准错误（stderr）。典型的如shell

```
// 进程定义
typedef struct proc
{
    spinlock_t lk; // 自旋锁

    /* 下面的六个字段需要持有锁才能修改 */

    int pid; // 标识符
    enum proc_state state; // 进程状态
    struct proc *parent; // 父进程
    int exit_state; // 进程退出时的状态(父进程可能关心)
    void *sleep_space;
    pgtbl_t pgtbl; // 用户态页表
    uint64 heap_top; // 用户堆顶(以字节为单位)
    uint64 ustack_pages; // 用户栈占用的页面数量
    mmap_region_t *mmap;
    trapframe_t *tf; // 用户态内核态切换时的运行环境暂存空间

    uint64 kstack; // 内核栈的虚拟地址
    context_t ctx; // 内核态进程上下文
    inode_t *cwd; // 当前目录
    file_t *filelist[FILE_PER_PROC]; // 打开文件列表
} proc_t;
```

```
xuke@VMware:~/os-lab/os-lab-temp$ ls -l /proc/$$/exe
lrwxrwxrwx 1 xuke xuke 0 12月 21 15:40 /proc/3452/exe -> /usr/bin/bash
```

说明当前 shell 进程（PID 3452）的可执行文件是 /usr/bin/bash

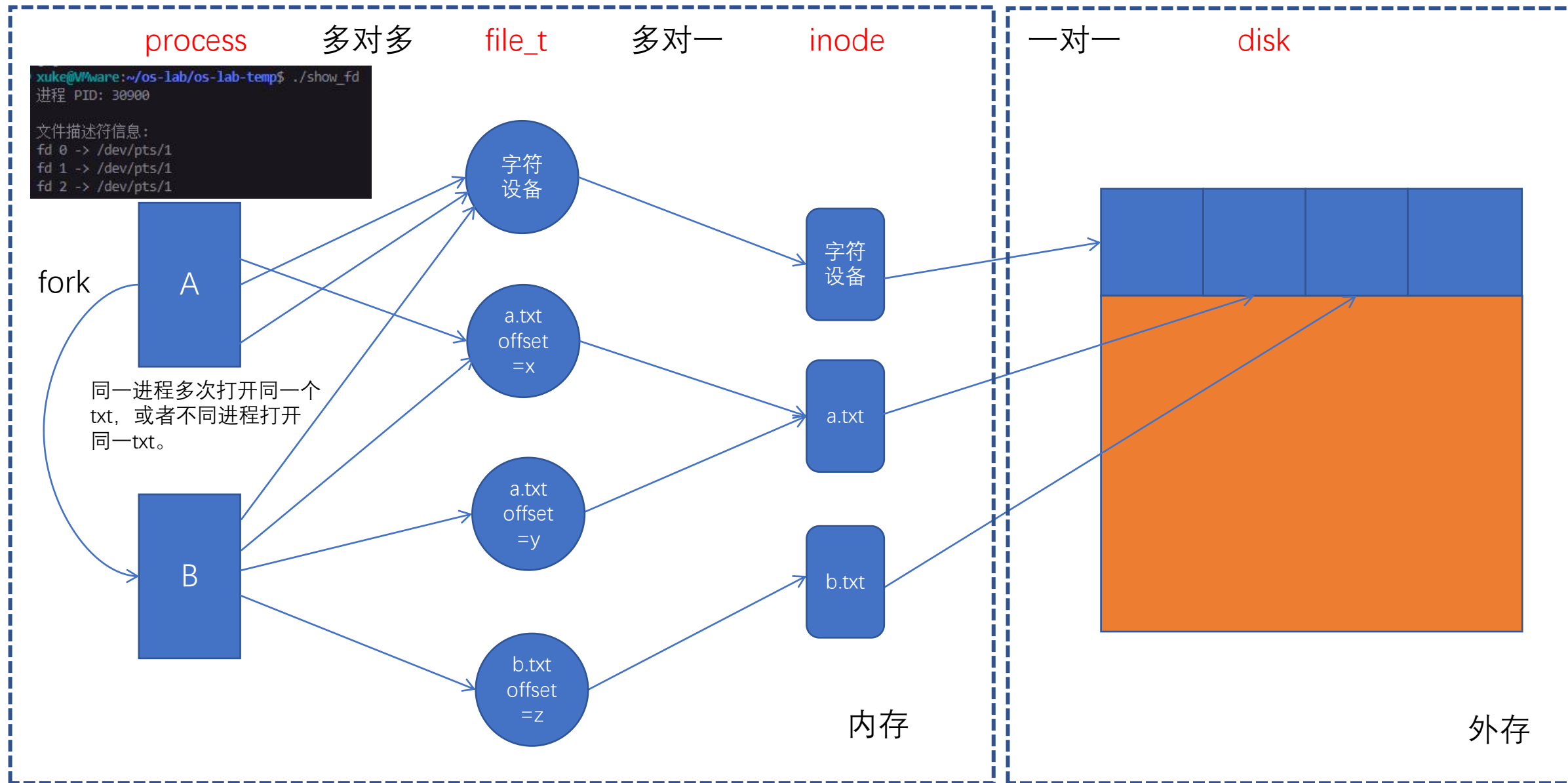
```
xuke@VMware:~/os-lab/os-lab-temp$ ls -l /proc/$$/fd/
总计 0
lrwx----- 1 xuke xuke 64 12月 21 12:24 0 -> /dev/pts/1
lrwx----- 1 xuke xuke 64 12月 21 12:24 1 -> /dev/pts/1
lr-x----- 1 xuke xuke 64 12月 21 12:24 17 -> /dev/urandom
lrwx----- 1 xuke xuke 64 12月 21 12:24 2 -> /dev/pts/1
l-wx----- 1 xuke xuke 64 12月 21 12:24 20 -> /home/xuke/.cursor-server/data/logs/20251221T122445/remoteagent.log
l-wx----- 1 xuke xuke 64 12月 21 12:24 21 -> /home/xuke/.cursor-server/data/logs/20251221T122445/ptyhost.log
lrwx----- 1 xuke xuke 64 12月 21 12:24 22 -> /dev/ptmx
lrwx----- 1 xuke xuke 64 12月 21 12:24 255 -> /dev/pts/1
```

fd 0, 1, 2, 255
/dev/pts/1
(伪终端从设备)

```
typedef struct file
{
    uint16 type; // 文件类型
    bool readable; // 可读?
    bool writable; // 可写?
    uint32 ref; // 引用数
    uint16 major; // 主设备号 (for device)
    uint32 offset; // 偏移量 (for file)
    inode_t *ip; // 对应的inode (for dir file device)
} file_t;
```

不同进程对同一个文件有自己的记录信息，比如读写偏移量。

1. 进程打开文件表



2. 设备文件

Linux 进程的核心操作或行为，主要可分为 “基于文件抽象IO操作（通过文件描述符）” 和 “基于内核接口的其他操作（不通过文件描述符，比如内存申请释放，fork等）”。但是最上层都是统一封装成系统调用。

进程的打开文件`file_t`能够进行io操作是因为对于设备文件能够索引到驱动。

在fork_return中给我们的0号进程默认打开了字符设备。

0号进程进入用户态fork出子进程执行stdout，传入打开的字符设备的fd

内核态通过fd找到`file_t`根据type判断是普通文件还是设备文件。

如果是普通文件就操作inode写入文件，如果是设备文件就用major索引驱动函数`devlist[major].write()`

```
// 准备标准输入和标准输出设备
file_t *file = file_create_dev(path: "/console", major: DEV_CONSOLE, minor: 0);
proczero->filelist[0] = file;
proczero->filelist[1] = file_dup(file);
proczero->cwd = path_to_inode(path: "/");
```

fork_return中打开文件

```
typedef struct file
{
    uint16 type; // 文件类型
    bool readable; // 可读?
    bool writable; // 可写?
    uint32 ref; // 引用数
    uint16 major; // 主设备号 (for device)
    uint32 offset; // 偏移量 (for file)
    inode_t *ip; // 对应的inode (for dir file device)
} file_t;
```

```
// 标准输出
uint32 stdout(char* str, uint32 len)
{
    return sys_write(fd: STD_OUT, len, addr: str);
}
```

```
// Linux内核使用cdev结构
struct cdev {
    struct kobject kobj;
    struct module *owner;
    const struct file_operations *ops; // 文件操作函数表
    struct list_head list;
    dev_t dev; // 设备号 (主设备号+次设备号)
    unsigned int count;
};
```

linux中字符设备的结构体

```
// 设备注册使用哈希表/链表，不是简单数组
// 通过主设备号索引到cdev_map哈希表
static struct kobj_map *cdev_map;
```

```
// 设备文件需要提供读写接口
typedef struct dev
{
    uint32 (*read)(uint32 len, uint64 dst, bool user_dst);
    uint32 (*write)(uint32 len, uint64 src, bool user_src);
} dev_t;
```


3. 用户库

在linux中用户库是“用户程序使用操作系统能力的标准入口层”。
同一可执行程序之所以跨系统无法运行，原因是因为他们syscall接口不同，所链接后端库实现也不同。

```
xuke@VMware:/dev/input$ ls /lib
android-sdk      girepository-1.0      locale
apg              git-core              lsb
apparmor         gnome-session         man-db
apt              gnome-settings-daemon-3.0  memtest86+
aspell           gnome-settings-daemon-46  mime
bfd-plugins     gnome-shell           modprobe.d
binfmt.d        gnupg                modules
brlrtty         gnupg2               modules-load.d
clang           gold-ld              netplan
cloud-init      groff                networkd-dispatcher
cnf-update-db  grub                 NetworkManager
command-not-found  grub-legacy          nvidia
compat-ld       gvfs                 openssh
console-setup   hdparm              openvpn
cpp            init                 openvswitch-switch
cups           initramps-tools      os-probes
dbus-1.0        ipxe                  os-release
debug          ispell                pam.d
dhcpcd         kernel                pcmciautils
dpkg           klibc                 pclock.d
dracut         klibc-sw0VaytFV0hmlGCQE9vyf0nJ81g.so  pkgconfig
emacs-common   ld-linux.so.2         pm-utils
environment.d  linux                 policykit-1
evolution-data-server  linux-boot-probes    polkit-1
file           linux-hwe-6.14-tools-6.14.0-35  pppd
firewalld      linux-hwe-6.14-tools-6.14.0-36  python3
firmware       linux-sound-base      python3.12
gcc            linux-tools           qemu
gcc-cross     llvm-18               recovery-mode
```

```
xuke@VMware:/dev/input$ ls /usr/include
aio.h      errno.h      gnu-versions.h  monetary.h  poll.h      shadow.h      ttyent.h
aliases.h  error.h      grp.h           mqueue.h    printf.h    signal.h      uchar.h
alloca.h   execinfo.h  gshadow.h      mtd          proc_service.h  sound        ucontext.h
argp.h     expat_external.h  iconv.h        net          pthread.h    spawn.h      ulimit.h
argz.h     expat.h      ifaddrs.h      netash       protocols    stab.h      unistd.h
ar.h       fcntl.h     inttypes.h     netatalk     pty.h        stdbit.h     utime.h
arpa       fdt.h        iproute2        netax25      pwd.h        stdc-predef.h  utmp.h
asm-generic  features.h   lastlog.h      netdb.h      python3.12   stdint.h     utmpx.h
assert.h    features-time64.h  libfdt_env.h  neteconet    rdma         stdio_ext.h  uuid
blkid       fenv.h       libfdt.h       netinet      re_comp.h    stdio.h      values.h
byteswap.h  finclude     libgen.h       netipx       regex.h      stdlib.h     video
c++         fmtmsg.h     libintl.h      netiucv      regexp.h     string.h     wait.h
clang       fnmatch.h    libmount       netpacket    regext.h     strings.h    wchar.h
complex.h   fstab.h      limits.h       netrom       resolv.h     sudo_plugin.h  wctype.h
cpio.h      fts.h        link.h         netrose      rpc          syscall.h     wordexp.h
crypt.h     ftw.h        linux          nfs           rpcsvc       syscalls.h    X11
ctype.h     gawkapi.h    locale.h      nl_types.h   sched.h      sysxits.h     x86_64-linux-gnu
dirent.h    gconv.h     malloc.h      nss.h        scsi          syslog.h      xen
dlfcn.h     getopt.h    math.h        obstack.h   search.h     tar.h         xorg
drm          getopt.h    mcheck.h     opendir.h   selinux      termio.h     xz
elf.h       gio-unix-2.0  memory.h     paths.h     semaphore.h  termios.h    zconf.h
endian.h    glob.h       misc          pcre2.h     sepol        tgmth.h      zlib.h
envz.h      gnumake.h    mntent.h     pcre2posix.h  setjmp.h     thread_db.h
err.h       gnu-headers  mntent.h     pixman-1     sgtty.h      time.h
```

对于我们本次项目的所有编译产物，除了mkfs使用了/lib当中的库，其他源文件的编译都是完全用我们自定义的头文件和库。

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <libgen.h>
#include <assert.h>
```

```
$(LD) ... -o $(TARGET_USER_PATH)/_test \
$(TARGET_USER_PATH)/test.o \
$(TARGET_USER_PATH)/user_lib.o \      # ← 项目自定义库
$(TARGET_USER_PATH)/user_syscall.o    # ← 项目自定义库
```

```
# 编译相关配置
CFLAGS = -Wall -Werror -O -fno-omit-frame-pointer -ggdb -gdwarf-2
CFLAGS += -MD
CFLAGS += -mcmodel=medany
CFLAGS += -ffreestanding -fno-common -nostdlib -mno-relax
CFLAGS += -I.
CFLAGS += $(shell $(CC) -fno-stack-protector -E -x c /dev/null >/dev/null 2>&1 && echo -fno-stack-protector)
```

链接选项，高度自定义

目录	内容
/lib/x86_64-linux-gnu/	系统核心库（libc, libm等）
/usr/lib/x86_64-linux-gnu/	用户库（应用库）
/lib/modules/	内核模块
/lib/firmware/	硬件固件
/lib/systemd/	systemd相关
/lib/gcc/	GCC工具链

3. 用户库

本次实验中我们在之前较为成熟的系统调用基础设施上仿照Linux封装许多自定义的用户api，然后编译成user_lib.o供跑在我们内核上的程序链接。

// 来自user_syscall.c

```
int sys_exec(char* path, char** argv);
uint64 sys_brk(uint64 new_heap_top);
uint64 sys_mmap(uint64 start, uint32 len);
uint64 sys_munmap(uint64 start, uint32 len);
int sys_fork();
int sys_wait(void* addr);
int sys_exit(int exit_state);
int sys_sleep(uint32 seconds);
int sys_open(char* path, uint32 open_mode);
int sys_close(int fd);
uint32 sys_read(int fd, uint32 len, void* addr);
uint32 sys_write(int fd, uint32 len, void* addr);
uint32 sys_lseek(int fd, uint32 offset, int flags);
int sys_dup(int fd);
int sys_fstat(int fd, fstat_t* state);
uint32 sys_getdir(int fd, dirent_t* addr, uint32 len);
int sys_mkdir(char* path);
int sys_chdir(char* path);
int sys_link(char* old_path, char* new_path);
int sys_unlink(char* path);
```

// 来自user_lib.c

```
void _main();
uint32 stdout(char* str, uint32 len);
uint32 stdin(char* str, uint32 len);
void memset(void* begin, uint8 data, uint32 n);
void memmove(void* dst, const void* src, uint32 n);
int strncmp(const char *p, const char *q, uint32 n);
int strlen(const char *str);
void printf(const char* fmt, ...);
void print_dirents(dirent_t* dir, uint32 count);
void print_filestate(fstat_t* file);

#endif
```

// 来自test.c

// 测试目的: 测试 stdin 和 stdout 输入一行然后回车换行

#include "userlib.h"

```
int main(int argc, char *argv[]) {
    char tmp[128];
    while (1) {
        memset(begin: tmp, data: 0, n: 128);
        stdin(str: tmp, len: 128);
        stdout(str: tmp, len: strlen(str: tmp));
    }
}
```

编译相关配置

```
CFLAGS = -Wall -Werror -O -fno-omit-frame-pointer -ggdb -gdwarf-2
CFLAGS += -MD
CFLAGS += -mmodel=medany
CFLAGS += -ffreestanding -fno-common -nostdlib -mno-relax
CFLAGS += -I.
CFLAGS += $(shell $(CC) -fno-stack-protector -E -x c /dev/null >/dev/null 2>&1 && echo -fno-stack-protector)
```

链接选项，高度自定义

编译user_lib.c生成user_lib.o

```
$(CC) $(CFLAGS) -I$(USER_PATH) -c $(USER_PATH)/user_lib.c -o $(TARGET_USER_PATH)/user_lib.o
```

编译user_syscall.c生成user_syscall.o

```
$(CC) $(CFLAGS) -I$(USER_PATH) -c $(USER_PATH)/user_syscall.c -o $(TARGET_USER_PATH)/user_syscall.o
```

先编译_test.c生成_test.o

```
$(CC) $(CFLAGS) -I$(USER_PATH) -c $(USER_PATH)/test.c -o $(TARGET_USER_PATH)/test.o
```

链接_test.o与用户库生成最终可执行文件_test

```
$(LD) $(LDFLAGS) -T $(USER_PATH)/user.ld -o $(TARGET_USER_PATH)/_test $(TARGET_USER_PATH)/test.o $(TARGET_USER_PATH)/user
```

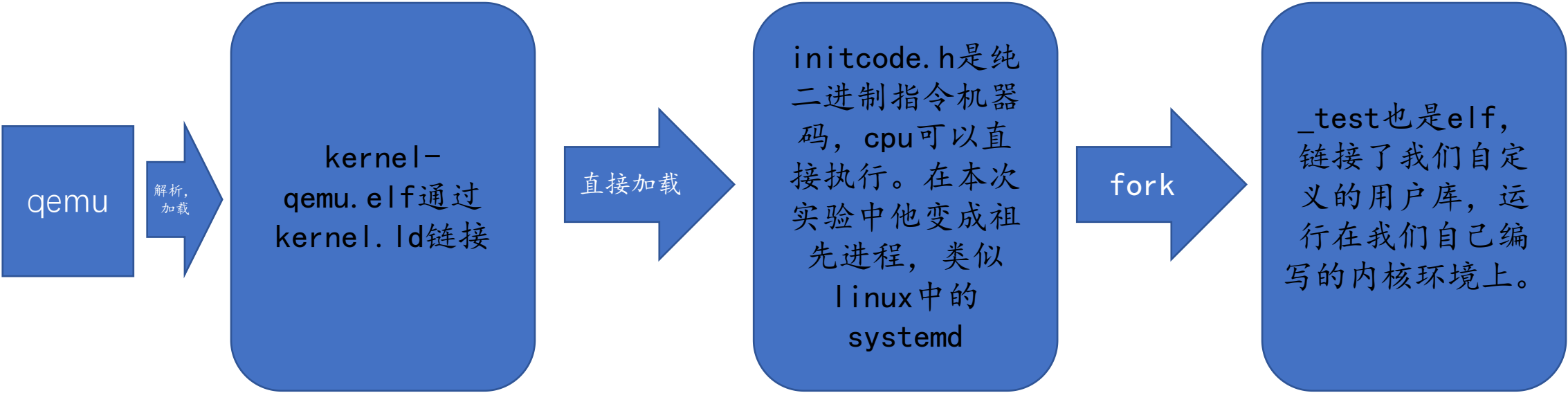
----- user process made

我们用自己编写的库供跑在我们自己编写的内核上的程序在编译的时候进行链接。这样我们就得到了一个唯一能够跑在我们自己内核上的可执行程序！——_test

4. 可执行文件

是一种被某个执行环境识别和解析，并能将其内容映射到内存，从而使CPU 开始取指执行的文件或数据载体。它的定义更加宽泛！

迄今为止我们项目中makefile编译了三个可执行文件：kernel-qemu.elf，initcode.h以及_test。
他们共同点在于都可以被cpu执行。不同点在于elf可执行文件是有格式的文件（可以被解析，更灵活），可以简单理解为对纯机器码做了一些管理和包装，而initcode.h只是纯机器码（cpu直接能执行）。为什么编译器链接器可以将源码变成cpu可执行的纯机器码，还要有elf这种？主要是让硬件处理器更专注于指令本身。解析的任务交给软件，不仅可以减轻硬件电路的复杂度，还有利于开发开发更高级的软件。



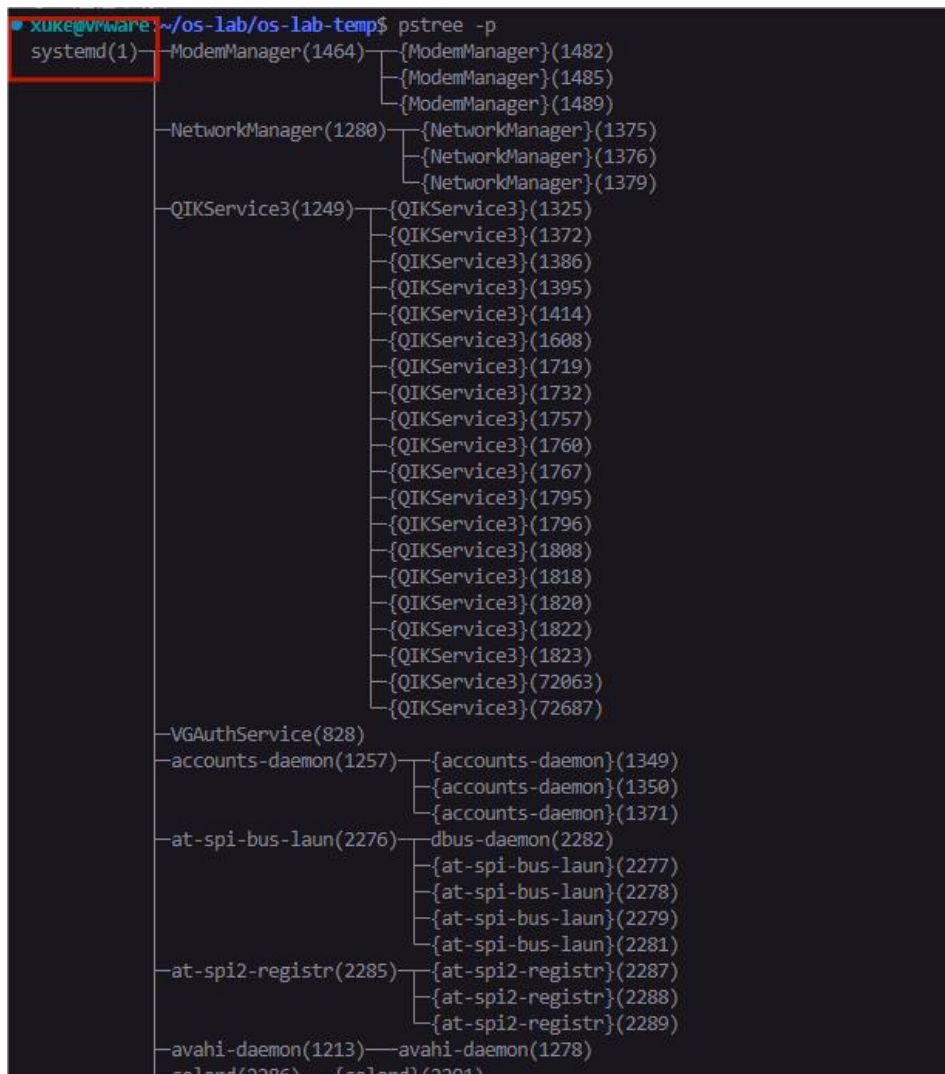
解析者	是否常见	解析ELF的目的
OS 内核 (execve)	☑	运行用户程序
动态链接器 (ld.so)	☑	装载共享库
QEMU	☑	启动内核 / 裸机
Bootloader (U-Boot)	☑	启动内核
Firmware (UEFI)	☑	启动 OS
调试器 (GDB)	☑	调试 / 断点
静态分析工具	☑	反汇编 / 检查
自定义 loader	☑	教学 / 实验
MCU / CPU	✗	永远不能

所以elf既然有结构，那必然有一个解析者，可以是os，qemu，分析工具等。
但不会是cpu或者MCU本身。所以在只有硬件没有内核的嵌入式开发中烧入的程序比如流水灯程序就只能是处理器认识的纯机器码！

```
xuke@VMware:~/os-lab/os-lab-temp$ sudo file /boot/vmlinuz-$(uname -r)
/boot/vmlinuz-6.14.0-36-generic: Linux kernel x86 boot executable bzImage, version 6.14.0-36-generic (buildd@lcy02-amd64-067) #36~24.04.1-Ubuntu
P PREEMPT_DYNAMIC Wed Oct 15 15:45:17 UTC 2, RO-rootFS, swap_dev 0XE, Normal VGA
```


4. 可执行文件

真实linux中的进程树，systemd是祖先进程。



没有用户态，内核代码的一部分，没有可执行文件。

内核态

systemd
前半生pid
为1

systemd
后半生pid
为1

进入用户态

0号进程idle，
负责调度启动1号进程

内核线程

挂载文件
系统

用elf替
换自己。

fork

其他进
程

fork不是复制吗？为什么
会产生其他不一样的进程？

调用 `execve("/sbin/init")`

```
xuke@VMware:~/os-lab/os-lab-temp$ file /sbin/init
/sbin/init: symbolic link to ../lib/systemd/systemd
```

我们的实验中简化了上述流程，将0号进程作为祖先进程，0号进程本身没有elf可执行文件而是直接执行initcode.h机器码。然后他作为祖先祖先进程来fork产生其他进程。

```
int pid = syscall(n: SYS_fork);
if (pid < 0) { // 失败
    syscall(n: SYS_write, 0, 20, "initcode: fork fail\n");
} else if (pid == 0) { // 子进程
    syscall(n: SYS_write, 0, 22, "\n-----test start-----\n");
    syscall(n: SYS_exec, path, argv);
} else { // 父进程
    syscall(n: SYS_wait, 0);
    syscall(n: SYS_write, 0, 21, "\n-----test over-----\n");
    while (1)
        ;
}
```

```
xuke@VMware:~/os-lab/os-lab-temp$ ./show_fd
进程 PID: 30900
```

文件描述符信息：
fd 0 -> /dev/pts/1
fd 1 -> /dev/pts/1
fd 2 -> /dev/pts/1

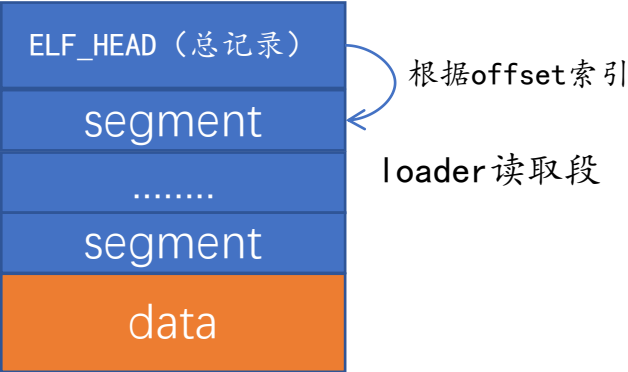
```
xuke@VMware:~/os-lab/os-lab-temp$ pid=$$; while true; do ps -p $pid -o pid=,ppid=,comm= 2>/dev/null
r -d ' '; [ -z "$pid" ] && break; done
183488    3086  bash
3086     3007  node
3007     2998  node
2998     2859  sh
2859          1  bash
1          0  systemd
```

vscode ssh连接虚拟机终端进程的祖先链

我们的shell也是同样的逻辑，当我们执行./hello.elf时，是shell进程通过fork的形式创建了子进程，子进程首先和shell长得完全一样，然后通过exec系统调用将文件系统中的hello.elf读取，解析，替换。最终夺舍，子进程继承了shell的空间，同时也将继承环境变量！export原理就是如此。

4. 可执行文件

可执行文件结构:



我们的elf_head

```
xuke@VMware:~/os-lab/os-lab-temp$ readelf -h target/user/_test
ELF 头:
  Magic:      7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  类别:      ELF64
  数据:      2 补码, 小端序 (little endian)
  Version:    1 (current)
  OS/ABI:     UNIX - System V
  ABI 版本:   0
  类型:      EXEC (可执行文件)
  系统架构:   RISC-V
  版本:      0x1
  入口点地址: 0x1048
  程序头起点: 64 (bytes into file)
  Start of section headers: 34016 (bytes into file)
  标志:      0x5, RVC, double-float ABI
  Size of this header: 64 (bytes)
  Size of program headers: 56 (bytes)
  Number of program headers: 3
  Size of section headers: 64 (bytes)
  Number of section headers: 18
  Section header string table index: 17
```

标准elf头结构定义:
无任何自定义。

```
// program header
typedef struct program_header {
    uint32 type;
    uint32 flags;
    uint64 off;
    uint64 va;
    uint64 pa;
    uint64 file_size;
    uint64 mem_size;
    uint64 align;
} program_header_t;
```

```
typedef struct elf_header {
    uint32 magic; // 应该是ELF_MAGIC
    uint8 elf[12]; // 一些信息
    uint16 type; // ELF文件类型(如可执行文件、共享库等)
    uint16 machine; // 机器的指令集架构(如x86、risc-v等)
    uint32 version; // ELF版本号
    uint64 entry; // 程序入口地址
    uint64 ph_off; // program header table偏移量
    uint64 sh_off; // section header table偏移量
    uint32 flags; // 处理器特定标志
    uint16 eh_size; // elf_header本身的大小
    uint16 ph_ent_size; // program header table里每个entry的大小
    uint16 ph_ent_num; // program header table里的entry数量
    uint16 sh_ent_size; // section header table里每个entry的大小
    uint16 sh_ent_num; // section header table里的entry数量
    uint16 sh_str_ndx; // section header table中包含节字符串表索引的entry的索引
} elf_header_t;
```

ELF 文件格式中 “整体描述” 与 “段加载信息” 的层级关系:
elf_header_t 是整个 ELF 文件的 “总目录”, 其中包含了 program_header_t 表的位置、大小、数量等关键信息, 通过这些信息可以找到并解析所有的 program_header_t 条目。

乌班图的elf_head

```
xuke@VMware:~/os-lab/os-lab-temp$ readelf -h hello
ELF 头:
  Magic:      7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  类别:      ELF64
  数据:      2 补码, 小端序 (little endian)
  Version:    1 (current)
  OS/ABI:     UNIX - System V
  ABI 版本:   0
  类型:      DYN (Position-Independent Executable file)
  系统架构:   Advanced Micro Devices X86-64
  版本:      0x1
  入口点地址: 0x1060
  程序头起点: 64 (bytes into file)
  Start of section headers: 13968 (bytes into file)
  标志:      0x0
  Size of this header: 64 (bytes)
  Size of program headers: 56 (bytes)
  Number of program headers: 13
  Size of section headers: 64 (bytes)
  Number of section headers: 31
  Section header string table index: 30
```

VS

4. 可执行文件

链接脚本. ld的差异导致最终可执行程序段数量差异。

```
1 OUTPUT_ARCH( "riscv" )
2 ENTRY( _main )
3
4 SECTIONS
5 {
6     . = 0x1000;
7
8     .text : {
9         *(.text .text.*)
10    }
11
12    .rodata : {
13        . = ALIGN(16);
14        *(.rodata .rodata.*)
15        . = ALIGN(16);
16        *(.rodata .rodata.*)
17        . = ALIGN(0x1000);
18    }
19
20    .data : {
21        . = ALIGN(16);
22        *(.data .data.*)
23        . = ALIGN(16);
24        *(.data .data.*)
25    }
26
27    .bss : {
28        . = ALIGN(16);
29        *(.bss .bss.*)
30        . = ALIGN(16);
31        *(.bss .bss.*)
32    }
33
34 PROVIDE(end = .);
```

VS

```
● xuke@VMware:~/os-lab/os-lab-temp$ ld --verbose
GNU ld (GNU Binutils for Ubuntu) 2.42
支持的仿真:
elf_x86_64
elf32_x86_64
elf_i386
elf_iamcu
i386pep
i386pe
使用内部链接脚本:
=====
/* Script for -z combrelloc -z separate-code */
/* Copyright (C) 2014-2024 Free Software Foundation, Inc.
Copying and distribution of this script, with or without modification,
are permitted in any medium without royalty provided the copyright
notice and this notice are preserved. */
OUTPUT_FORMAT("elf64-x86-64", "elf64-x86-64",
               "elf64-x86-64")
OUTPUT_ARCH(i386:x86-64)
ENTRY(_start)
SEARCH_DIR("/usr/local/lib/x86_64-linux-gnu"); SEARCH_DIR("/lib/x86_64-linux-
"); SEARCH_DIR("/usr/lib/x86_64-linux-gnu64"); SEARCH_DIR("/usr/local/lib64
b64"); SEARCH_DIR("/usr/local/lib"); SEARCH_DIR("/lib"); SEARCH_DIR("/usr/
"); SEARCH_DIR("/usr/x86_64-linux-gnu/lib");
SECTIONS
{
    PROVIDE (__executable_start = SEGMENT_START("text-segment", 0x400000)); . =
OF_HEADERS;
```

真实乌班图链接脚本
300+行

```
xuke@VMware:~/os-lab/os-lab-temp$ objdump -s -j .text target/user/_test
target/user/_test: 文件格式 elf64-little

Contents of section .text:
1000 757106e5 22e10009 13060008 81451305 uq.....E..
1010 04f79700 0000e780 00149305 00081305 .....
1020 04f79700 0000e780 20111305 04f79700 .....
1030 0000e780 401a9b05 05001305 04f79700 .....@.....
1040 0000e780 a002c9b7 411106e4 22e00008 .....A.....
1050 97000000 e78000fb 97000000 e780c04b .....K.....
1060 a2600264 41018280 411106e4 22e00008 ..dA.....
1070 2a860145 97000000 e7800051 0125a260 *.E.....Q.%`
1080 02644101 82803971 06fc22f8 26f44af0 .dA...9q..".&.J.
1090 800019c2 63420508 01258148 230c04fc .....cB...%.H#...
10a0 930674fd 3d478125 17060000 13060669 ..t.=G.%.....i
10b0 3a887d37 bb77b502 82178193 b29783c7 :.)7.w.....
10c0 07002380 f6009b07 05003b55 b502fd16 ..#.....;U....
10d0 e3f0b7fe 638c0800 930707fe 33878700 .....c.....3....
10e0 9307d002 2304f7fe 1b07e8ff 9b041700 .....#.....
10f0 1b890400 63460902 c145859d 930784fc .....cF...E.....
1100 33852701 97000000 e78040f6 e2704274 3.'.....@..pBt
1110 a2740279 21618280 3b05a040 8548bdfb .t.yla...;.@.H..
1120 b9451705 00001305 e5529700 0000e780 .E.....R.....
1130 e0f3d9b7 411106e4 22e00008 2a860545 .....A.....*.E
1140 97000000 e780c042 0125a260 02644101 .....B.%`.dA..
```

我们的机器码更多
是因为静态链接，
乌班图的hello是动
态链接

VS

```
● xuke@VMware:~/os-lab/os-lab-temp$ objdump -s -j .text hello
hello: 文件格式 elf64-x86-64

Contents of section .text:
1060 f30f1efa 31ed4989 d15e4889 e24883e4 ....1.I..^H..H..
1070 f0505445 31c031c9 488d3dca 000000ff .PTE1.1.H.=....
1080 15532f00 00f4662e 0f1f8400 00000000 .S/...f.....
1090 488d3d79 2f000048 8d05722f 00004839 H.=y/.H..r/..H9
10a0 f8741548 8b05362f 00004885 c07409ff .t.H..6/..H..t..
10b0 e00f1f80 00000000 c30f1f80 00000000 .....
10c0 488d3d49 2f000048 8d35422f 00004829 H.=I/.H.5B/..H)
10d0 fe4889f0 48c1ee3f 48c1f803 4801c648 .H..H..?H...H.H
10e0 d1fe7414 488b0505 2f000048 85c07408 .t.H.../..H..t.
10f0 ffe0660f 1f440000 c30f1f80 00000000 ...f..D.....
1100 f30f1efa 803d052f 00000075 2b554883 ...../...u+UH.
1110 3de22e00 00004889 e5740c48 8b3de62e =...H..t.H.=..
1120 0000e819 ffffffff 64ffffff c05dd2e .....d.....
1130 0000015d c30f1f00 c30f1f80 00000000 ...].....
1140 f30f1efa e977ffff fff30f1e fa554889 .....w.....UH.
1150 e5488d3d ac0e0000 e8f3feff ffb80000 .H.=.....
1160 00005dc3 .....]
```

我们的脚本
30+行

4. 可执行文件

可执行文件中定义的映射内存情况。

```
xuke@VMware:~/os-lab/os-lab-temp$ readelf -l target/user/_test

Elf 文件类型为 EXEC (可执行文件)
Entry point 0x1048
There are 3 program headers, starting at offset 64

程序头:
Type           Offset             VirtAddr           PhysAddr
FileSiz        MemSiz              Flags             Align
RISCV_ATTRIBUT 0x0000000000007cd8 0x0000000000000000 0x0000000000000000
0x000000000000053 0x0000000000000000 R                  0x1
LOAD            0x0000000000001000 0x0000000000001000 0x0000000000001000
0x0000000000001500 0x0000000000001500 RWE               0x1000
GNU_STACK       0x0000000000000000 0x0000000000000000 0x0000000000000000
0x0000000000000000 0x0000000000000000 RW                0x10

Section to Segment mapping:
段节...
00 .riscv.attributes
01 .text .rodata .eh_frame .data
02
```

VS

```
xuke@VMware:~/os-lab/os-lab-temp$ readelf -l hello

Elf 文件类型为 DYN (Position-Independent Executable file)
Entry point 0x1060
There are 13 program headers, starting at offset 64

程序头:
Type           Offset             VirtAddr           PhysAddr
FileSiz        MemSiz              Flags             Align
PHDR            0x0000000000000040 0x0000000000000040 0x0000000000000040
0x00000000000002d8 0x00000000000002d8 R                  0x8
INTERP          0x0000000000000318 0x0000000000000318 0x0000000000000318
0x000000000000001c 0x000000000000001c R                  0x1
[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
LOAD            0x0000000000000000 0x0000000000000000 0x0000000000000000
0x0000000000000628 0x0000000000000628 R                  0x1000
LOAD            0x0000000000001000 0x0000000000001000 0x0000000000001000
0x000000000000171 0x000000000000171 R E               0x1000
LOAD            0x0000000000002000 0x0000000000002000 0x0000000000002000
0x0000000000000f4 0x0000000000000f4 R                  0x1000
LOAD            0x0000000000002db8 0x0000000000003db8 0x0000000000003db8
0x000000000000258 0x000000000000260 RW               0x1000
DYNAMIC          0x0000000000002dc8 0x0000000000003dc8 0x0000000000003dc8
0x0000000000001f0 0x0000000000001f0 RW                  0x8
NOTE            0x000000000000338 0x000000000000338 0x000000000000338
0x000000000000030 0x000000000000030 R                  0x8
NOTE            0x000000000000368 0x000000000000368 0x000000000000368
0x000000000000044 0x000000000000044 R                  0x4
GNU_PROPERTY     0x000000000000338 0x000000000000338 0x000000000000338
0x000000000000030 0x000000000000030 R                  0x8
GNU_EH_FRAME     0x0000000000002014 0x0000000000002014 0x0000000000002014
0x000000000000034 0x000000000000034 R                  0x4
GNU_STACK        0x0000000000000000 0x0000000000000000 0x0000000000000000
0x000000000000000 0x000000000000000 RW                  0x10
GNU_RELRO        0x0000000000002db8 0x0000000000003db8 0x0000000000003db8
0x000000000000248 0x000000000000248 R                  0x1

Section to Segment mapping:
段节...
00
01 .interp
02 .interp .note.gnu.property .note.gnu.build-id .note.ABI-tag .gnu.hash .dynsym .dynstr .gnu.version
.rela.dyn
03 .init .plt .plt.got .plt.sec .text .fini
04 .rodata .eh_frame_hdr .eh_frame
05 .init_array .fini_array .dynamic .got .data .bss
06 .dynamic
07 .note.gnu.property
08 .note.gnu.build-id .note.ABI-tag
09 .note.gnu.property
10 .eh_frame_hdr
11
12 .init_array .fini_array .dynamic .got
```

LAOD标签的segment，是要被os内核中的段加载函数映射到用户地址空间中的，而且虚拟地址pa已经给出，size也在program_head中给出。

.text 起始地址 = 0x1000
_main 实际地址 = 0x1048 ←
readelf 显示的 entry point



4. 可执行文件

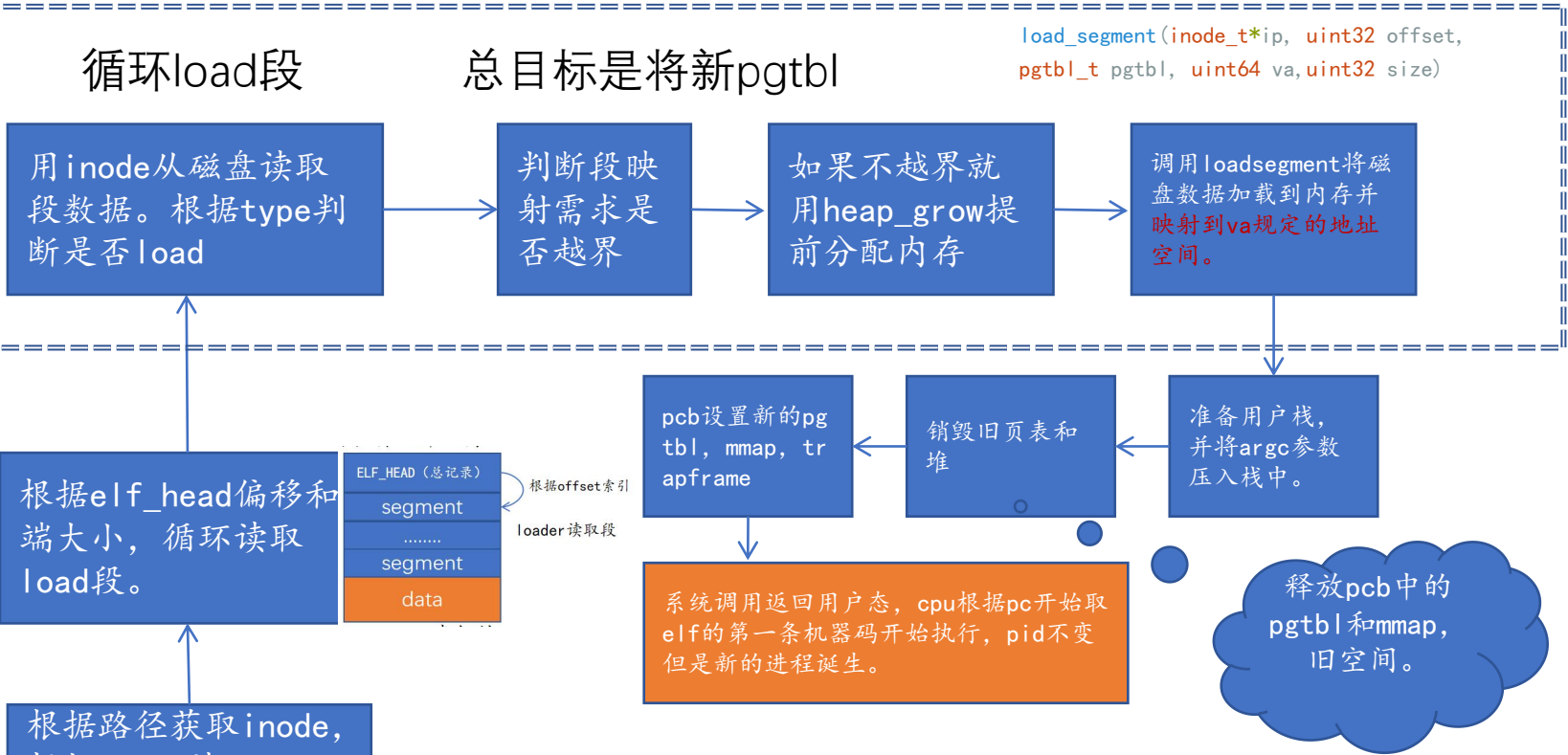
exec系统调用流程（夺舍流程）。

```
int pid = syscall(n: SYS_fork);
if (pid < 0) { // 失败
    syscall(n: SYS_write, 0, 20, "initcode: fork fail\n");
} else if (pid == 0) { // 子进程
    syscall(n: SYS_write, 0, 22, "\n-----test start-----\n");
    syscall(n: SYS_exec, path, argv);
} else { // 父进程
    syscall(n: SYS_wait, 0);
    syscall(n: SYS_write, 0, 21, "\n-----test over-----\n");
    while (1)
        ;
}
```

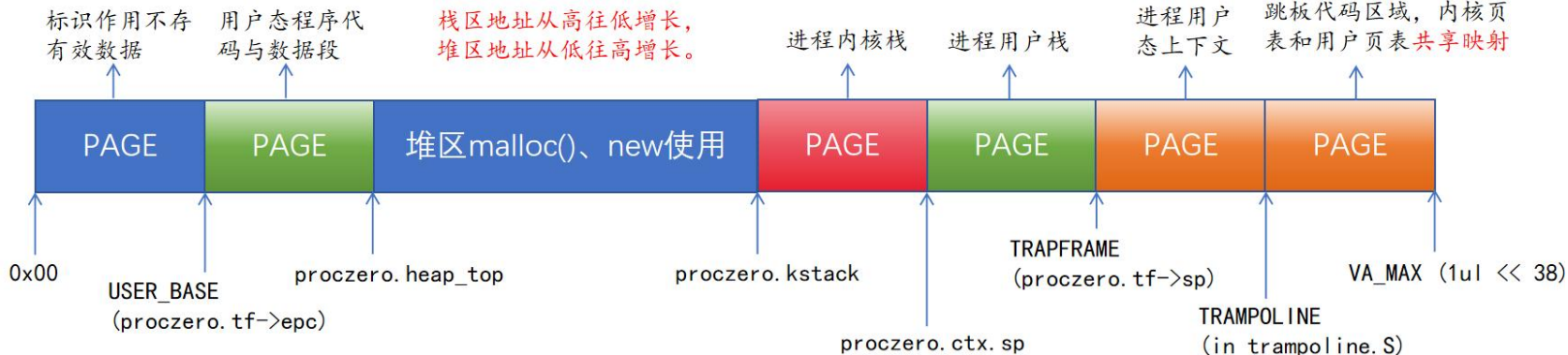
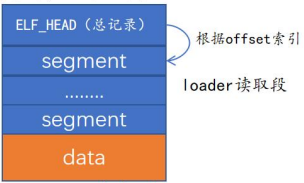
类比git
push等
操作

子进程系统调用
`int proc_exec(char *path, char **argv)`
接下来的目的就是完善新的
pgtbl

新建页表pgtbl，因为此时子进程本身在内核态执行还需要自己的内核栈空间来执行proc_exec，并不是直接占用。



根据路径获取inode，根据inode读取elf_head数据确定魔数。



```
// program header
typedef struct program_header {
    uint32 type;
    uint32 flags;
};

typedef struct elf_header {
    uint32 magic; // 应该是ELF_MAGIC
    uint8 elf[12]; // 一些信息
    uint16 type; // ELF文件类型(如可执行文件、共享库等)
    uint16 machine; // 机器的指令集架构(如x86、risc-v等)
    uint32 version; // ELF版本号
    uint64 entry; // 程序入口地址
    uint64 ph_off; // program header table偏移量
    uint64 sh_off; // section header table偏移量
    uint32 flags; // 处理器特定标志
    uint16 eh_size; // elf_header本身的大小
    uint16 ph_ent_size; // program header table里每个entry的大小
    uint16 ph_ent_num; // program header table里的entry数量
    uint16 sh_ent_size; // section header table里每个entry的大小
    uint16 sh_ent_num; // section header table里的entry数量
    uint16 sh_str_ndx; // section header table中包含节字符串表索引的entry的索引
} elf_header_t;
```

5. 总结

- 祝大家实验顺利，能够从底层开始一步一步输出一个真正的hello world!